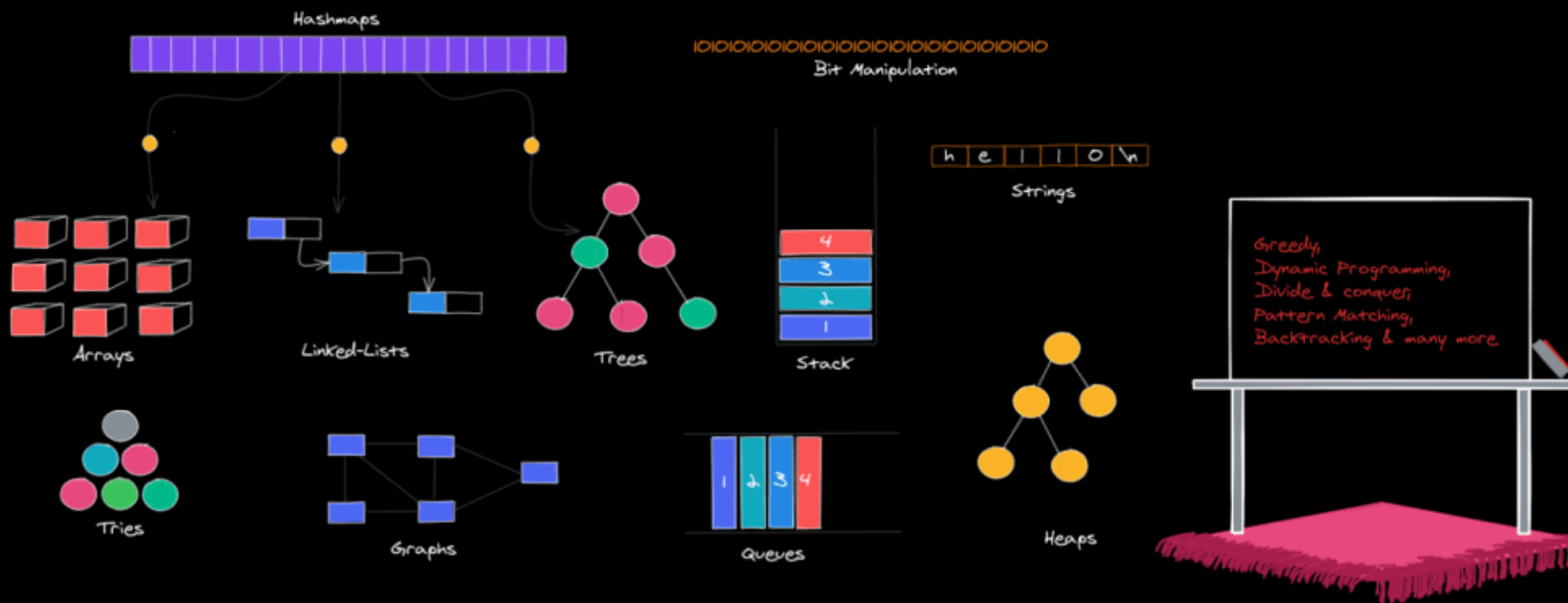




数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

数组



抽象数据类型 (ADT) 回顾

- 抽象数据类型 (ADT) 是一种数据结构的抽象化表达方式
- 一个ADT包括:
 - 存储的数据对象
 - 数据上的操作

数组的ADT

ADT Array {

数据对象: $A = \{A_1, A_2, \dots, A_N\}$

基本操作:

A.init(S): 使用序列S初始化数组A

len(A): 返回数组A的长度

A.is_empty(): 检查数组A是否为空, 如为空则返回True

A.get_item(n): 返回数组A的第n个元素

A.locate(e): 返回数组A中第一个等于e的元素的位置

A.insert(n,e): 在数组A的第n个元素前插入元素e

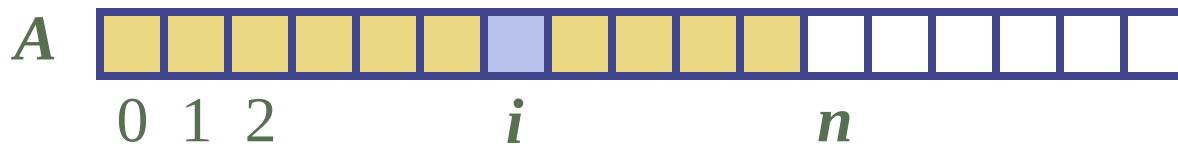
A.delete(n): 删除数组A的第n个元素

A.clear(): 清空数组A

} ADT Array

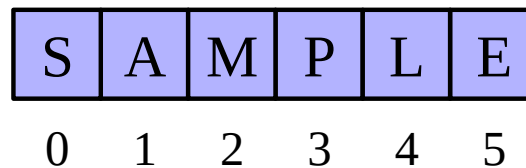
Python序列类型

- Python有各种序列类型，即内置的列表（list），元组（tuple），和字符串（str）类
- 这些序列类型支持用下标访问序列元素，如A[i]
- 这些类型均使用数组来表示序列
 - 一个数组即为一组相关变量，它们一个接一个地存在内存里的一块连续区域内

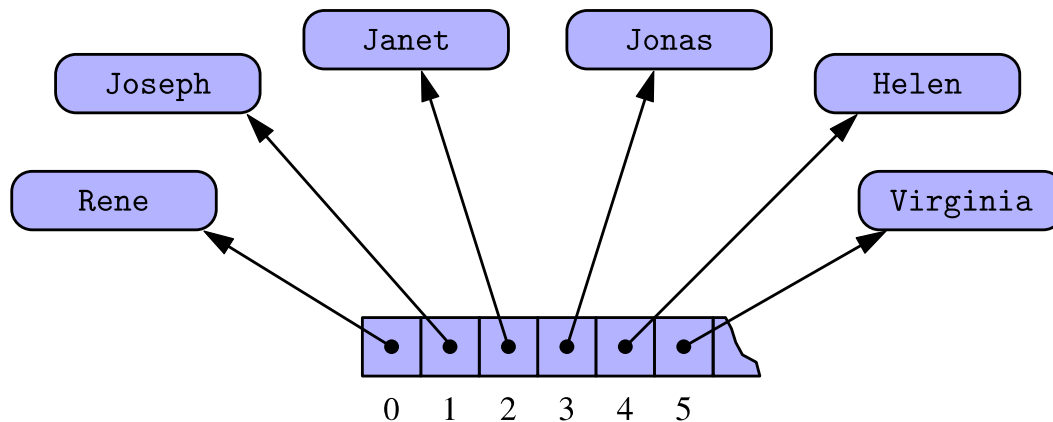


紧凑数组和引用数组

- 一个存储基本元素（如字符）的数组，被称为**紧凑数组**



- 一个存储对象的引用的数组，被称为**引用数组**



紧凑数组

- 紧凑数组主要由array模块提供支撑
 - array模块定义了一个array类，该类为基本数据类型提供紧凑数组的方法
- array类的构造函数需要以类型代码作为第一个参数（字符），该字符表明要存入数组的数据类型

```
primes = array('i', [2, 3, 5, 7, 11, 13, 17, 19])
```

array类支持的类型代码

Code	C Data Type	Typical Number of Bytes
'b'	signed char	1
'B'	unsigned char	1
'u'	Unicode char	2 or 4
'h'	signed short int	2
'H'	unsigned short int	2
'i'	signed int	2 or 4
'I'	unsigned int	2 or 4
'l'	signed long int	4
'L'	unsigned long int	4
'f'	float	4
'd'	float	8

动态数组

- 在一个 *add(o)* 操作中（无索引），我们将新元素附加到原数组尾部
- 当数组已满时，我们将当前数组换成更大的一个数组
 - 固定增量策略：每次数组大小增大一个常数 c
 - 翻倍增量策略：每次数组大小翻倍

```
Algorithm add(o):  
  if  $n = S.length - 1$  then  
     $A \leftarrow$  new array of  
      size ...  
    for  $i \leftarrow 0$  to  $n-1$  do  
       $A[i] \leftarrow S[i]$   
     $S \leftarrow A$   
     $n \leftarrow n + 1$   
     $S[n-1] \leftarrow o$ 
```

策略比较

- 我们分析总运行时间 $T(n)$ 来比较固定增量策略和翻倍增量策略对一个长为 n 的序列进行 $add(o)$ 操作
- 假设我们从一个空数组开始
- 我们称 add 运算的**摊销时间**为一个 add 运算在数组上运行的平均时间，即 $T(n)/n$

固定增量策略分析

- 我们初始化数组次数为 $k = n/c$ 次
- n 次 *add* 操作消耗的总时间 $T(n)$ 与下列式子成正比:

$$n + c + 2c + 3c + 4c + \dots + kc =$$

$$n + c(1 + 2 + 3 + \dots + k) =$$

$$n + ck(k + 1)/2$$

- 由于 c 为常数, $T(n)$ 为 $O(n + k^2)$, 即 $O(n^2)$
- *add* 操作的摊销时间复杂度为 $O(n)$

翻倍增量策略分析

- 我们初始化了 $k = \log_2 n$ 次数组
- n 次 *add* 操作消耗的总时间 $T(n)$ 与下列式子成正比:

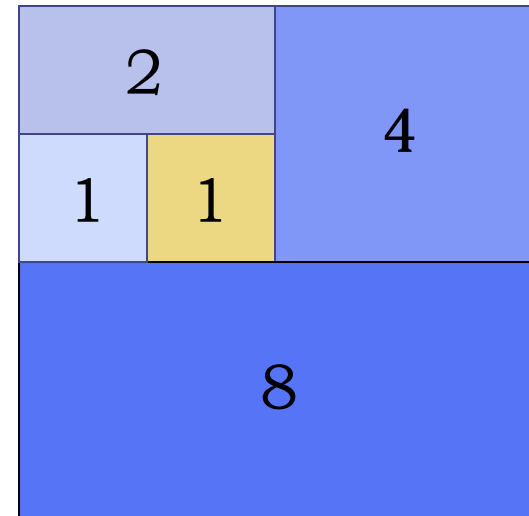
$$n + 1 + 2 + 4 + 8 + \dots + 2^k =$$

$$n + 2^{k+1} - 1 =$$

$$3n - 1$$

- $T(n)$ 为 $O(n)$
- *add* 操作的摊销时间复杂度为 $O(1)$

几何级数



Python中的动态数组

- Python中的list即为动态数组，它关联一个底层数组，其长度通常比list更长
- 通过运行如下代码可看到list底层数组长度的变化过程

```
import sys                                # provides getsizeof function
data = []
for k in range(n):                        # NOTE: must fix choice of n
    a = len(data)                          # number of elements
    b = sys.getsizeof(data)                # actual size in bytes
    print('Length: {0:3d}; Size in bytes: {1:4d}'.format(a, b))
    data.append(None)                     # increase length by one
```

Python中的动态数组

• 代码运行结果:

Length: 0; Size in bytes: 56	Length: 17; Size in bytes: 248
Length: 1; Size in bytes: 88	Length: 18; Size in bytes: 248
Length: 2; Size in bytes: 88	Length: 19; Size in bytes: 248
Length: 3; Size in bytes: 88	Length: 20; Size in bytes: 248
Length: 4; Size in bytes: 88	Length: 21; Size in bytes: 248
Length: 5; Size in bytes: 120	Length: 22; Size in bytes: 248
Length: 6; Size in bytes: 120	Length: 23; Size in bytes: 248
Length: 7; Size in bytes: 120	Length: 24; Size in bytes: 248
Length: 8; Size in bytes: 120	Length: 25; Size in bytes: 312
Length: 9; Size in bytes: 184	Length: 26; Size in bytes: 312
Length: 10; Size in bytes: 184	Length: 27; Size in bytes: 312
Length: 11; Size in bytes: 184	Length: 28; Size in bytes: 312
Length: 12; Size in bytes: 184	Length: 29; Size in bytes: 312
Length: 13; Size in bytes: 184	Length: 30; Size in bytes: 312
Length: 14; Size in bytes: 184	Length: 31; Size in bytes: 312
Length: 15; Size in bytes: 184	Length: 32; Size in bytes: 312
Length: 16; Size in bytes: 184	Length: 33; Size in bytes: 376
	Length: 34; Size in bytes: 376
	Length: 35; Size in bytes: 376
	Length: 36; Size in bytes: 376

Python实现动态数组

```
import ctypes                                # provides low-level arrays

class DynamicArray:
    """A dynamic array class akin to a simplified Python list."""

    def __init__(self):
        """Create an empty array."""
        self._n = 0                           # count actual elements
        self._capacity = 1                   # default array capacity
        self._A = self._make_array(self._capacity) # low-level array

    def _make_array(self, c):
        """Return new array with capacity c.""" # nonpublic utility
        return (c * ctypes.py_object)()       # see ctypes documentation

    def __len__(self):
        """Return number of elements stored in the array."""
        return self._n

    def __getitem__(self, k):
        """Return element at index k."""
        if not 0 <= k < self._n:
            raise IndexError('invalid index')
        return self._A[k]                    # retrieve from array
```

Python实现动态数组

```
def append(self, obj):  
    """Add object to end of the array."""  
    if self._n == self._capacity:  
        self._resize(2 * self._capacity)    # not enough room  
                                              # so double capacity  
    self._A[self._n] = obj  
    self._n += 1  
  
def _resize(self, c):  
    """Resize internal array to capacity c."""    # nonpublic utility  
    B = self._make_array(c)    # new (bigger) array  
    for k in range(self._n):    # for each existing value  
        B[k] = self._A[k]  
    self._A = B    # use the bigger array  
    self._capacity = c
```


Python中的列表和元组

- 列表和元组不改变其内容的操作：
 - k 表示被搜索值第一次出现位置的索引，或两个长度为 n_1 和 n_2 的序列比较时第一个不符合条件的位置或 $\min(n_1, n_2)$

操作	运行时间
<code>len(data)</code>	
<code>data[j]</code>	
<code>data.count(value)</code>	
<code>data.index(value)</code>	
<code>value in data</code>	
<code>data1 == data2</code> (similarly <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>)	
<code>data[j:k]</code>	
<code>data1 + data2</code>	
<code>c * data</code>	

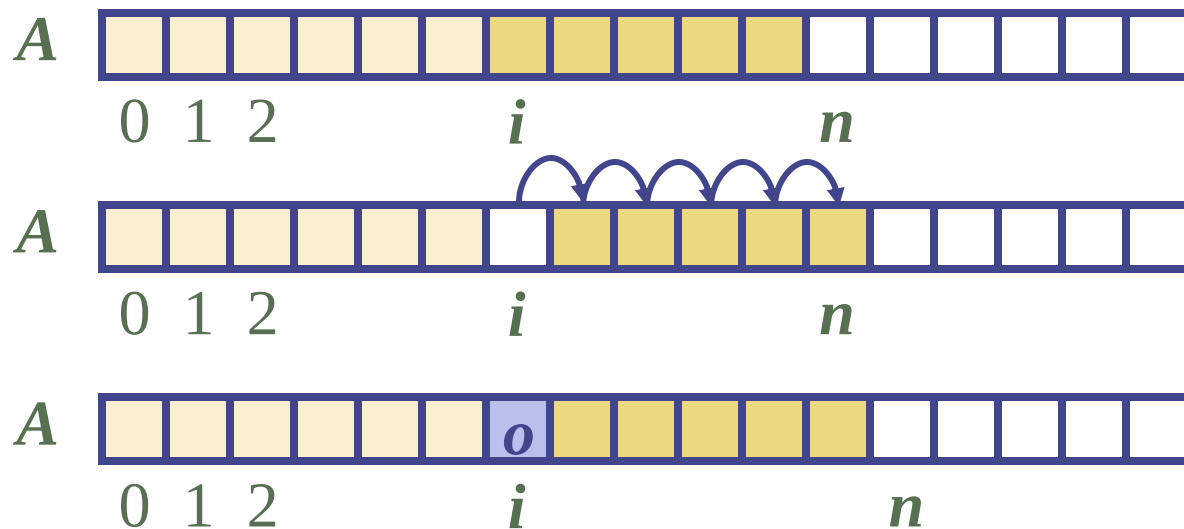
Python中的列表和元组

- 列表和元组不改变其内容的操作：
 - k 表示被搜索值第一次出现位置的索引，或两个长度为 n_1 和 n_2 的序列比较时第一个不符合条件的位置或 $\min(n_1, n_2)$

操作	运行时间
<code>len(data)</code>	$O(1)$
<code>data[j]</code>	$O(1)$
<code>data.count(value)</code>	$O(n)$
<code>data.index(value)</code>	$O(k)$
<code>value in data</code>	$O(k)$
<code>data1 == data2</code> (similarly <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>)	$O(k)$
<code>data[j:k]</code>	$O(k-j)$
<code>data1 + data2</code>	$O(n_1+n_2)$
<code>c * data</code>	$O(cn)$

数组的元素插入

- 在插入操作 $\text{insert}(i, o)$ 中，为了给新元素腾出位置，我们需要将数组的后 $n - i$ 个元素 $A[i], \dots, A[n - 1]$ 往后移1个位置
- 最坏情况下($i = 0$)，这需要 $O(n)$ 的时间

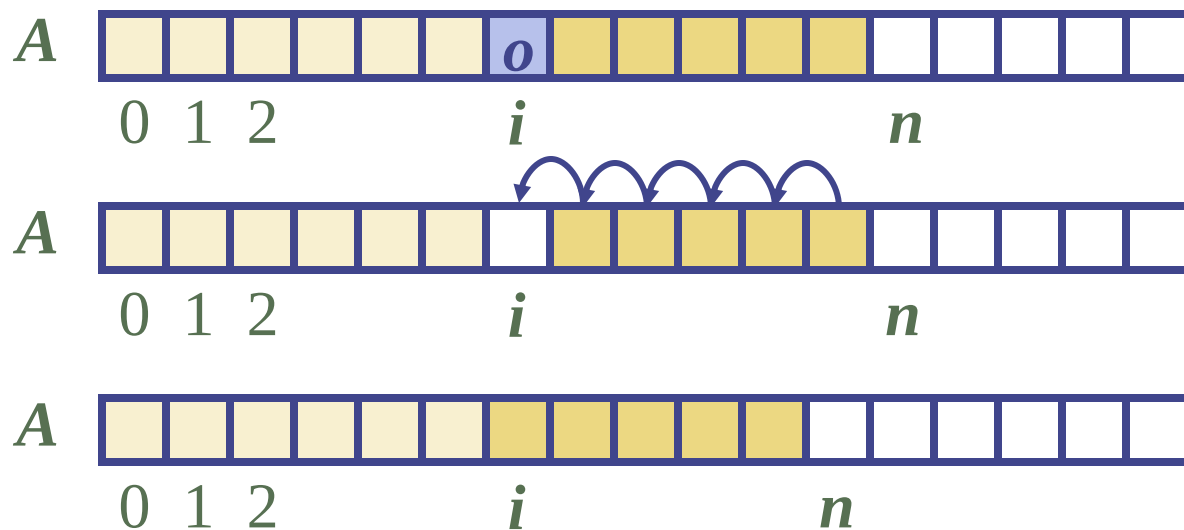


Python动态数组的插入操作

```
def insert(self, k, value):
    """Insert value at index k, shifting subsequent values rightward."""
    # (for simplicity, we assume 0 <= k <= n in this version)
    if self._n == self._capacity:                # not enough room
        self._resize(2 * self._capacity)         # so double capacity
    for j in range(self._n, k, -1):               # shift rightmost first
        self._A[j] = self._A[j-1]
    self._A[k] = value                            # store newest element
    self._n += 1
```

数组的元素删除

- 在删除操作`delete(i)`中，我们需要将 $n - i - 1$ 个元素 $A[i + 1], \dots, A[n - 1]$ 往前移1位以填补空缺
- 在最坏情况下($i = 0$)这需要 $O(n)$ 的时间



Python动态数组的删除操作

```
def remove(self, value):
    """Remove first occurrence of value (or raise ValueError)."""
    # note: we do not consider shrinking the dynamic array in this version
    for k in range(self._n):
        if self._A[k] == value:
            # found a match!
            for j in range(k, self._n - 1):
                # shift others to fill gap
                self._A[j] = self._A[j+1]
            self._A[self._n - 1] = None
            # help garbage collection
            self._n -= 1
            # we have one less item
            return
            # exit immediately
    raise ValueError('value not found')
    # only reached if no match
```

复杂度分析

- 在一个基于数组实现的动态列表中：
 - 此数据结构的空间复杂度为 $O(n)$
 - 取某个位置 i 对应的值的空间复杂度为 $O(1)$
 - 最坏情况下 *insert* 和 *remove* 的时间复杂度为 $O(n)$
- 在 *insert* 操作中，当数组已满时，我们可以使用一个更大的数组替换掉它，而非抛出异常（使用动态数组）
- 实际上，python中的list在删除时也会动态调整长度

插入排序

- 排序：将无序序列变为有序
- 一种友好简单的排序算法——插入排序
 - 插入第 i 个元素时，前 $i - 1$ 个元素已排好序，用第 i 个元素关键字和前 $i - 1$ 个元素关键字进行比较，确定插入位置

Algorithm InsertionSort(A):

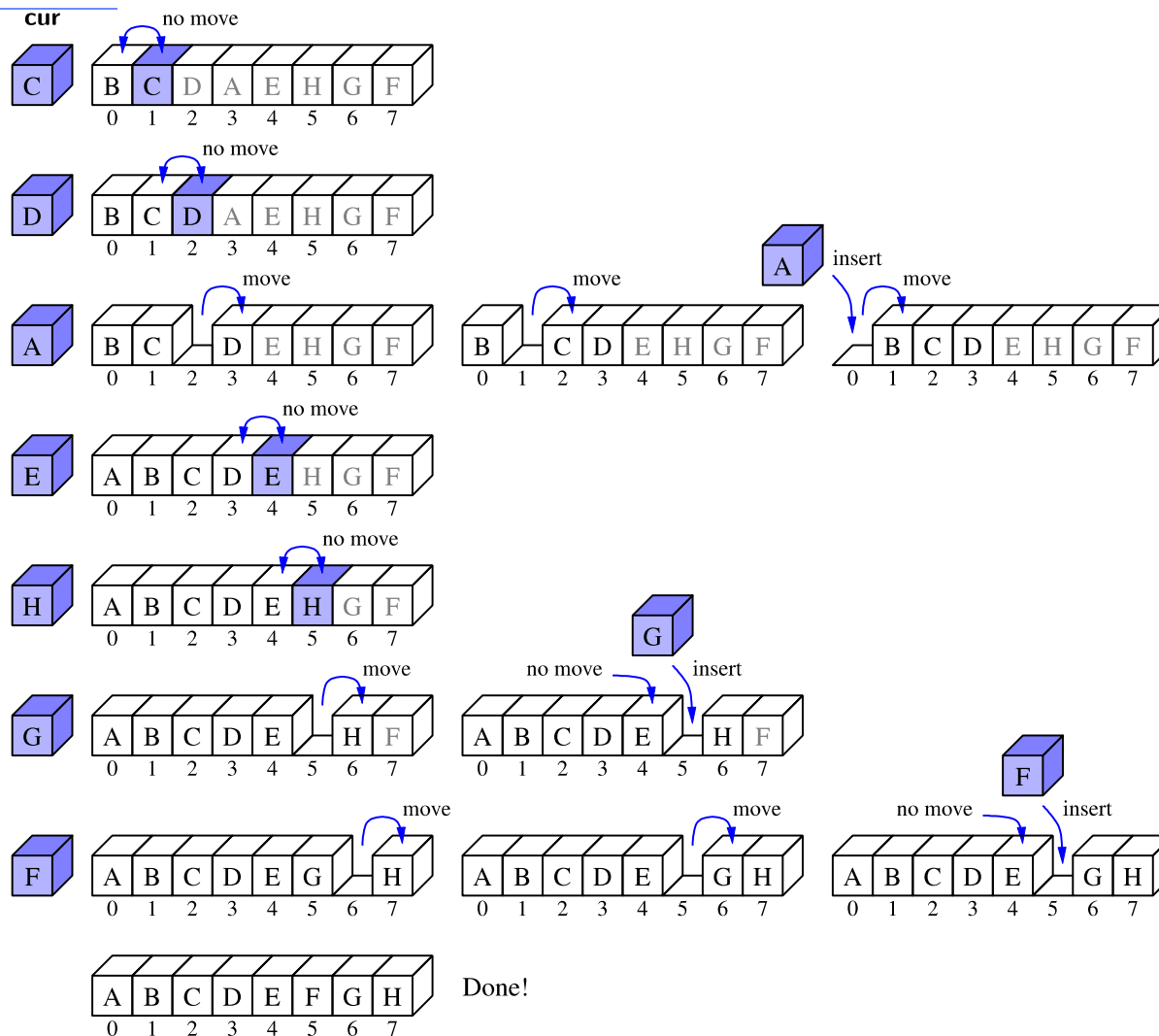
Input: An array A of n comparable elements

Output: The array A with elements rearranged in nondecreasing order

for k from 1 to $n - 1$ **do**

 Insert $A[k]$ at its proper location within $A[0], A[1], \dots, A[k]$.

插入排序



插入排序

- Python中插入排序的实现:

```
def insertion_sort(A):  
    """Sort list of comparable elements into nondecreasing order."""  
    for k in range(1, len(A)):          # from 1 to n-1  
        cur = A[k]                      # current element to be inserted  
        j = k                           # find correct index j for current  
        while j > 0 and A[j-1] > cur:   # element A[j-1] must be after current  
            A[j] = A[j-1]  
            j -= 1  
        A[j] = cur                      # cur is now in the right place
```

插入排序

- 插入排序的运行时间：
 - 最坏情况：初始数组为反序，则工作量最大，运行时间 $O(n^2)$
 - 最好情况：初始数组基本排序或已完全排序，则运行时间 $O(n)$
 - 平均情况： $O(n^2)$

Python中的列表和元组

- 列表改变其内容的操作：
 - n, n_1, n_2 表示 `data, data1, data2` 的长度

操作	运行时间
<code>data[j] = val</code>	
<code>data.append(value)</code>	
<code>data.insert(k, value)</code>	
<code>data.pop()</code>	
<code>data.pop(k)</code> <code>del data[k]</code>	
<code>data.remove(value)</code>	
<code>data1.extend(data2)</code> <code>data1 += data2</code>	
<code>data.reverse()</code>	
<code>data.sort()</code>	

Python中的列表和元组

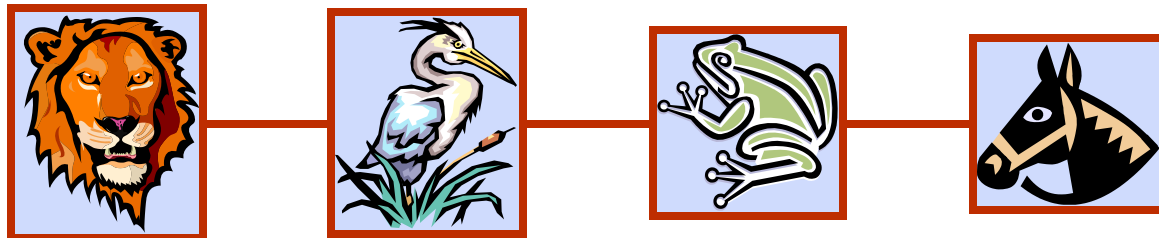
- 列表改变其内容的操作：
 - n, n_1, n_2 表示 `data, data1, data2` 的长度

操作	运行时间
<code>data[j] = val</code>	$O(1)$
<code>data.append(value)</code>	$O(1)^*$
<code>data.insert(k, value)</code>	$O(n - k)^*$
<code>data.pop()</code>	$O(1)^*$
<code>data.pop(k)</code> <code>del data[k]</code>	$O(n - k)^*$
<code>data.remove(value)</code>	$O(n)^*$
<code>data1.extend(data2)</code> <code>data1 += data2</code>	$O(n_2)^*$
<code>data.reverse()</code>	$O(n)$
<code>data.sort()</code>	$O(n \log n)$

*摊销



链表

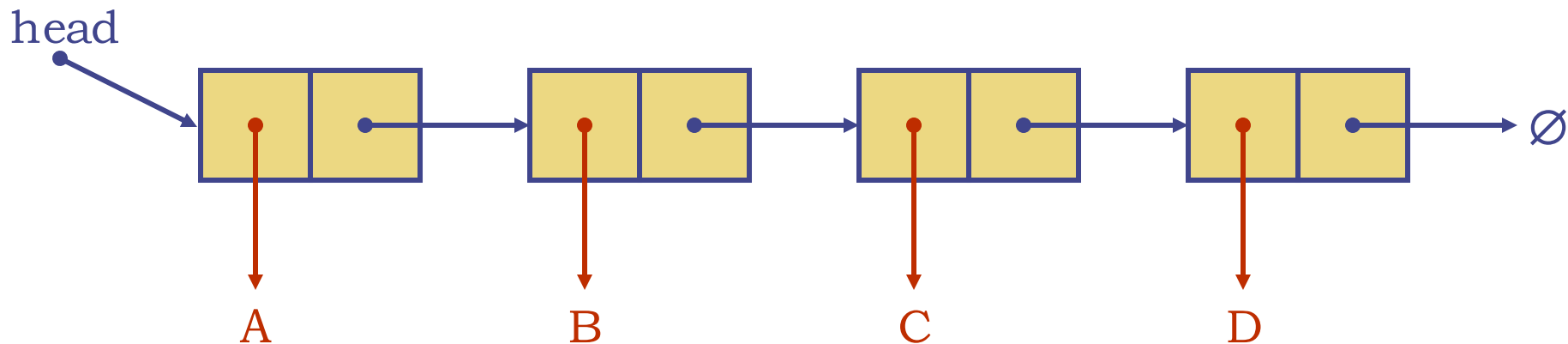
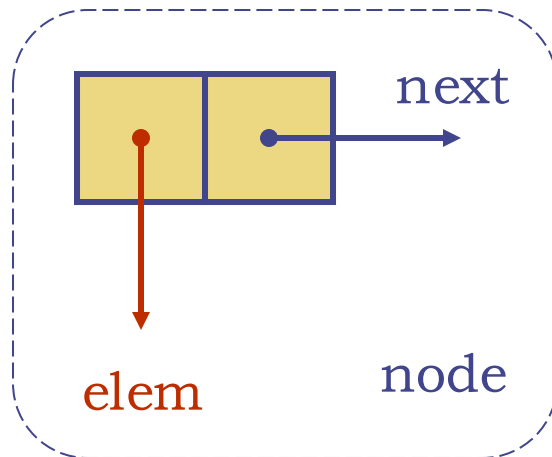


链表

- Python的list类是高度优化的，但也有一些明显的缺点：
 - 底层动态数组的长度超过实际存储数组元素所需的长度
 - 在实时系统中，被摊销的一些操作的运行时间是不可接受的
 - 数组内部插入、删除的代价太高
- 链表：
 - 依赖于分布式表达方法，采用节点，存放数据的存储单元不连续
 - 插入、删除非常方便

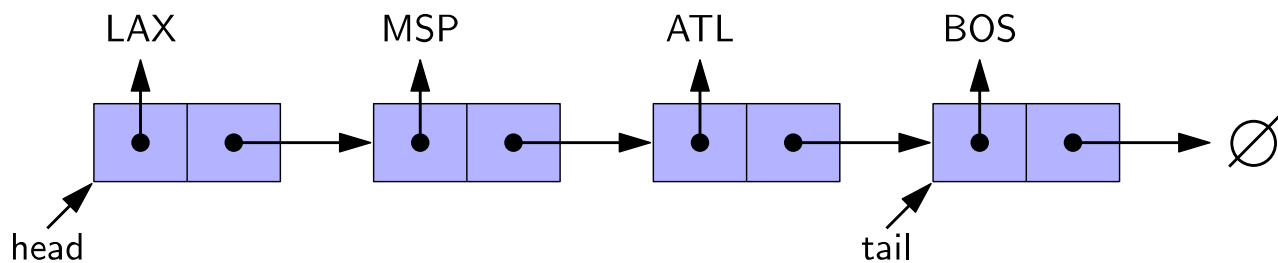
单向链表

- 单向链表是由多个节点共同构成的一个线性序列，从头指针开始
- 每个节点储存：
 - 元素值
 - 指向下一节点的指针



单向链表

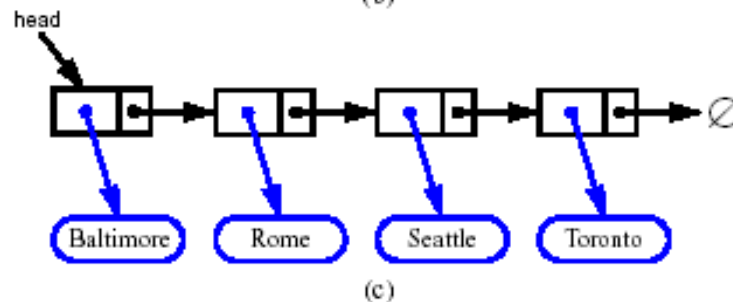
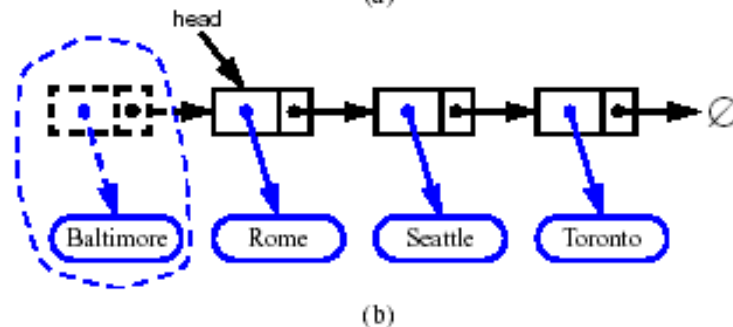
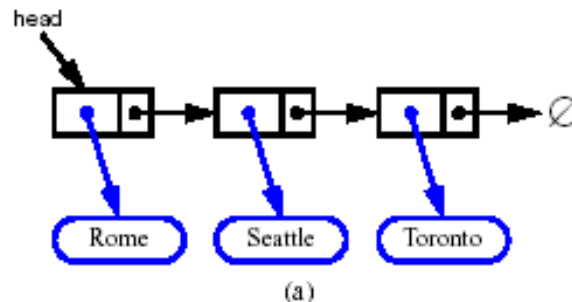
- 链表的第一个节点为**头节点**，最后一个节点为**尾节点**
- 头指针**head指向头节点，**尾指针**tail指向尾节点，且尾节点指向下一节点的指针为空指针
- 从头指针开始，通过每个节点的next指针可以到达下一个节点，直至到达尾指针，完成对链表的遍历



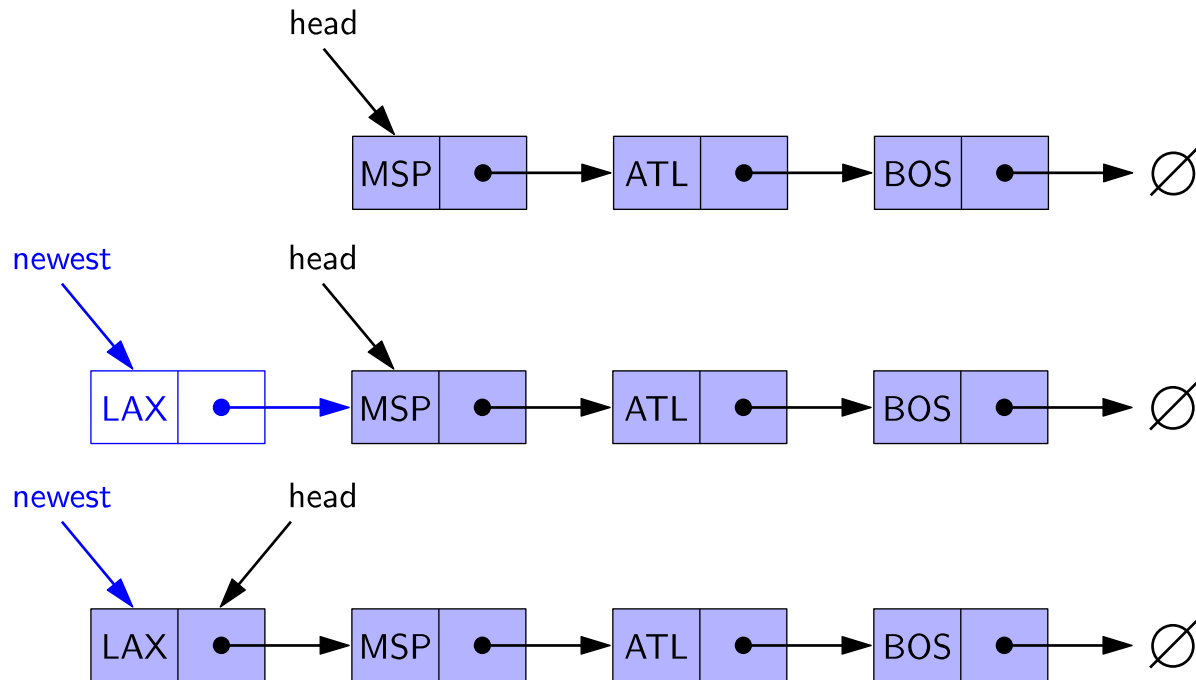
在头部插入元素

1. 创建一个新节点
2. 让新节点指向原来的头节点
3. 让头指针指向新节点

Algorithm add_first(L, e):
 newest = Node(e)
 newest.next = L.head
 L.head = newest
 L.size = L.size + 1



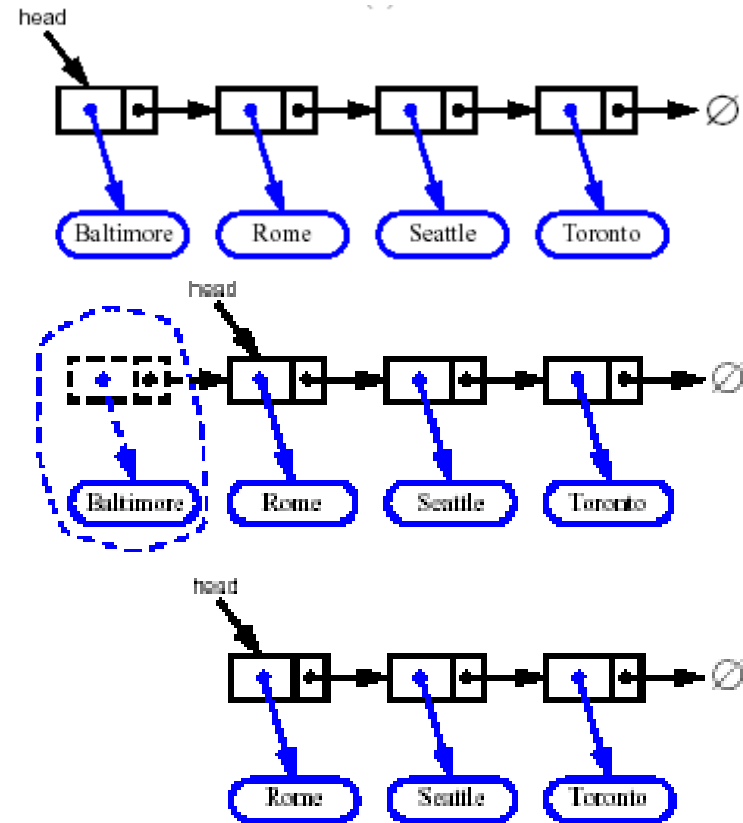
在头部插入元素



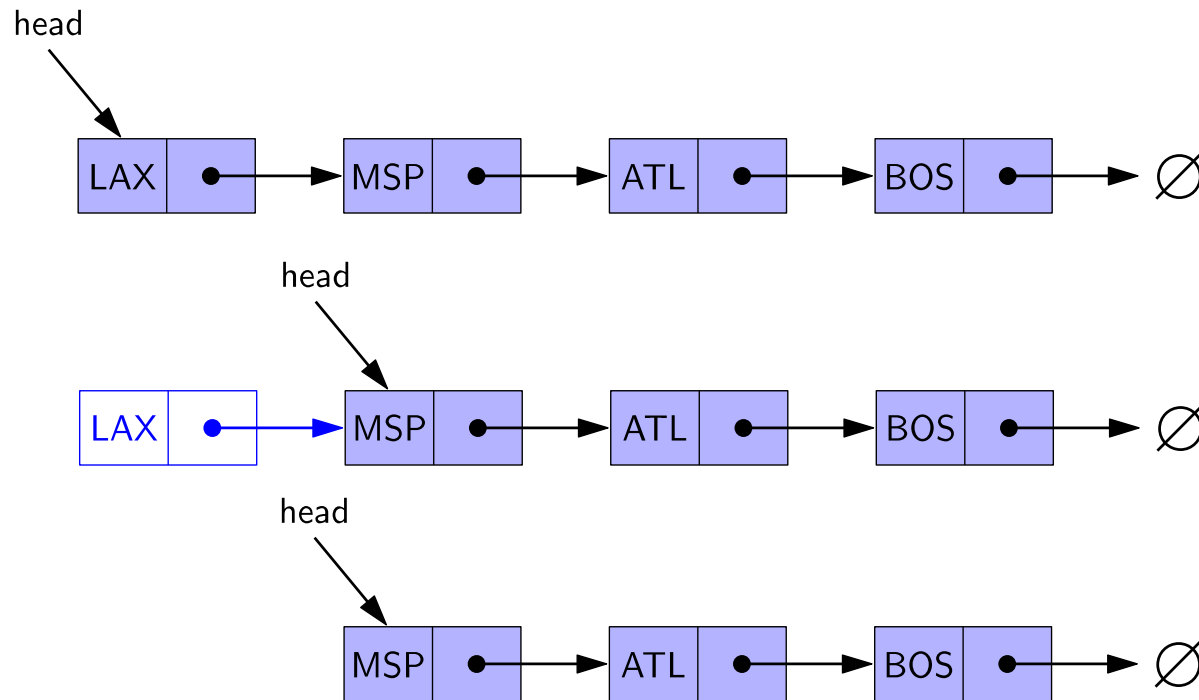
在头部删除元素

1. 记下原来的节点
2. 将头指针指向头节点的下一个节点
3. 释放原来的头节点

Algorithm remove_first(L):
if L.head is None **then**
 Indicate an error: the list is empty.
 p = L.head
 L.head = L.head.next
 delete(p)
 L.size = L.size - 1



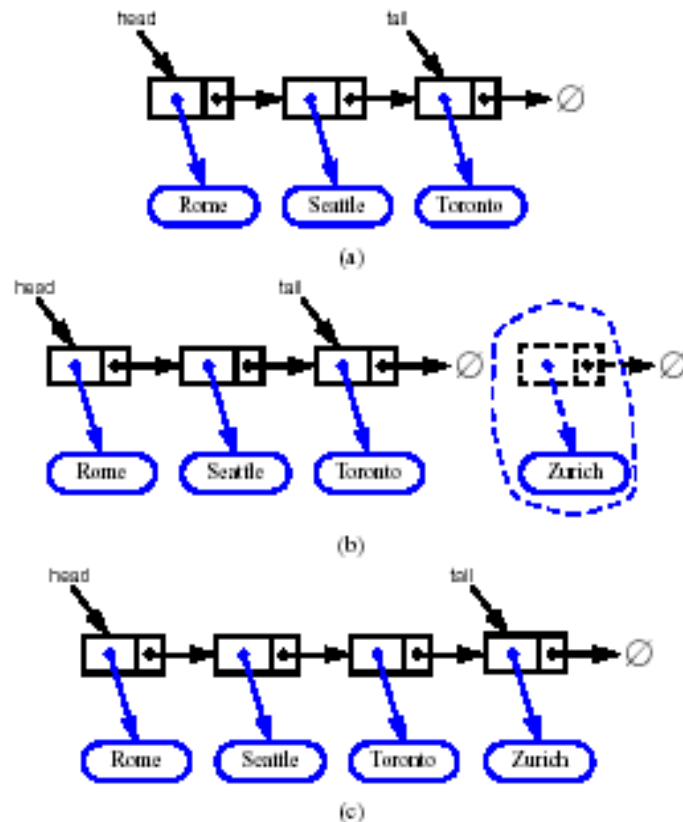
在头部删除元素



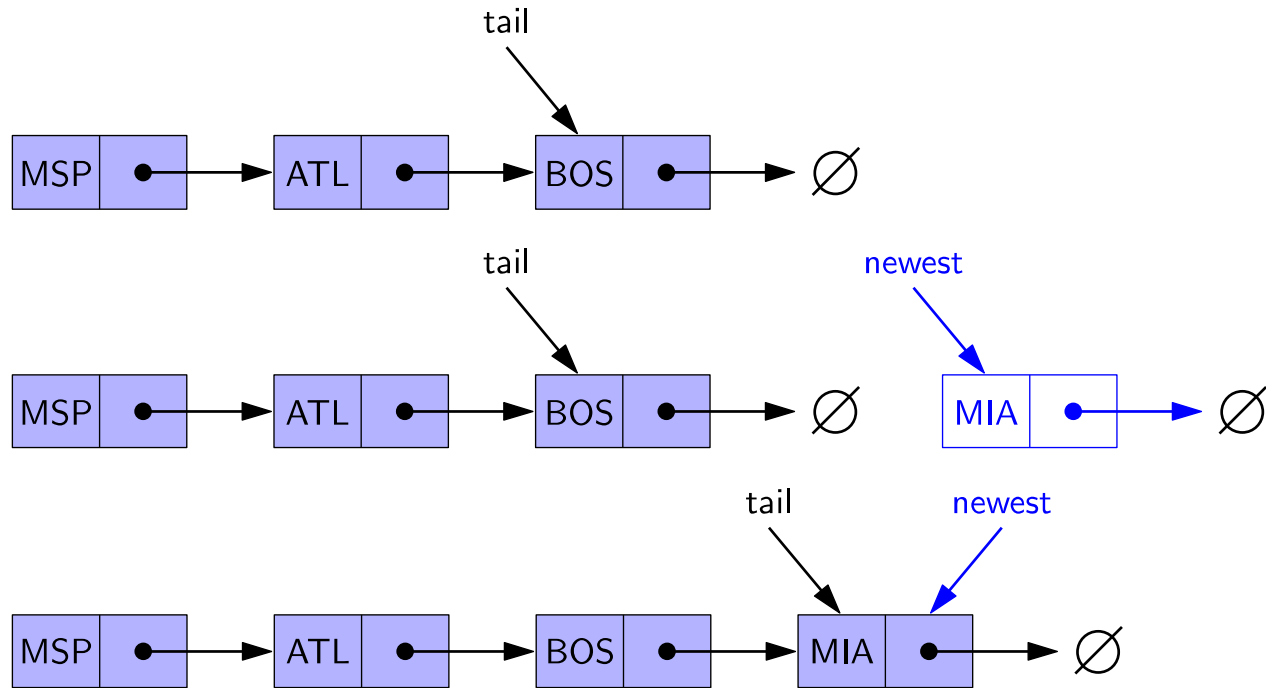
在尾部插入元素

1. 创建一个新节点
2. 将新节点指向空
3. 将原来的尾节点指向新节点
4. 将尾指针指向新节点

Algorithm add_last(L, e):
 newest = Node(e)
 newest.next = None
 L.tail.next = newest
 L.tail = newest
 L.size = L.size + 1

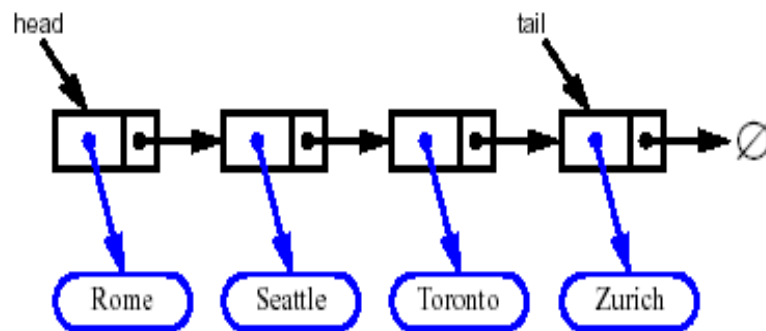


在尾部插入元素



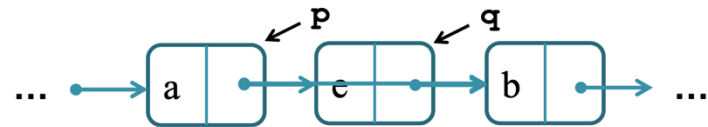
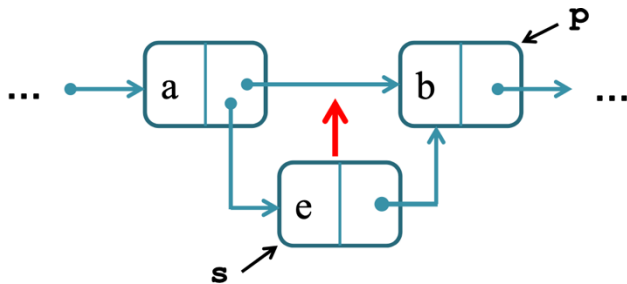
在尾部删除元素

- 删除链表尾部的元素是很低效的
- 没有常数时间内可删除尾节点的方法
- 时间复杂度为?



一般位置的插入与删除

- 思考：如何在一般位置（非头、尾）进行插入和删除？
 - 插入：将新元素e插入到第m个位置上（遵循Python的零索引）
 - 删除：将第m个位置上的节点删除（遵循Python的零索引）



一般位置的插入与删除

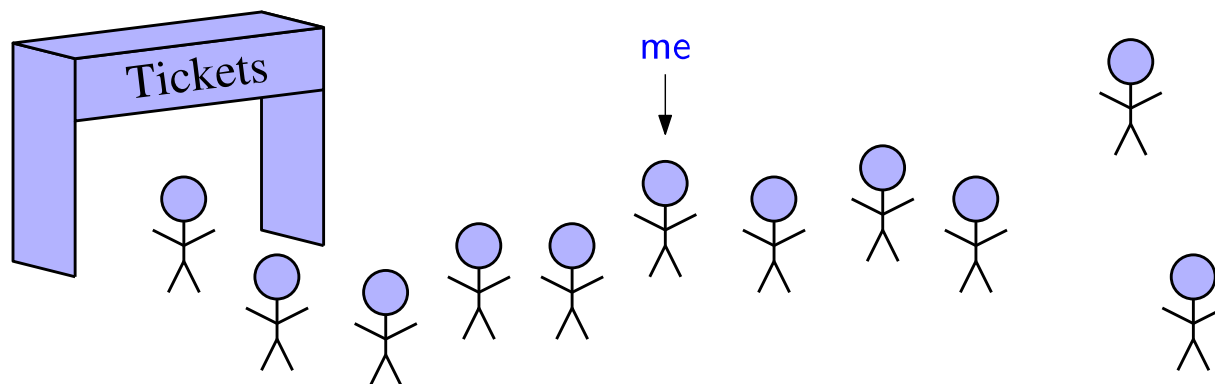
- 思考：如何在一般位置（非头、尾）进行插入和删除？
 - 插入：将新元素e插入到第m个位置上（遵循Python的零索引）
 - 删除：将第m个位置上的节点删除（遵循Python的零索引）

Algorithm insert(L, m, e):

Algorithm remove(L, m):

单向链表的优势

- 描述位置时，索引并非一个好的选择——同一位置的索引值可能发生变化
- 例：
 - 排队者的具体位置
 - 文本处理器中，使用游标描述文档中的位置
- 链表的优势：节点实际上标明了其存储元素值所在的**位置**，此位置不会随其他节点的变化而发生变动



单向链表的节点类

```
class _Node:
    """Lightweight, nonpublic class for storing a singly linked node."""
    __slots__ = '_element', '_next'           # streamline memory usage

    def __init__(self, element, next):        # initialize node's fields
        self._element = element              # reference to user's element
        self._next = next                    # reference to next node
```

单向链表的实现

```
class LinkedList:
    """A base class providing a linked list representation."""

    # ----- nested _Node class -----
    class _Node:
        """Lightweight class for storing a linked list node."""
        __slots__ = '_element', '_next' # streamline memory

        def __init__(self, element, next): # initialize node's fields
            self._element = element # user's element
            self._next = next # next node reference

    # ----- list constructor -----
    def __init__(self):
        """Create an empty list."""
        self._head = None
        self._size = 0 # number of elements
```

单向链表的实现

```
# ----- accessors -----  
  
def first(self):  
    """Return the first position in the list (or None if list is empty)."""  
    return self._head  
  
def after(self, p):  
    """Return the position just after position p (or None if p is last)."""  
    return p._next  
  
def get_node(self, n):  
    """Return the n-th node (position) of the list."""  
    if n >= self._size:  
        raise IndexError('Index out of bound!')  
    cur = self._head  
    for k in range(n):  
        cur = cur._next  
    return cur
```

单向链表的实现

```
def get_pos_element(self, p):
    """Return the element at position p."""
    return p._element

def get_ind_element(self, n):
    """Return the n-th element of the list."""
    cur = self.get_node(n)
    return cur._element

def __iter__(self):
    """Generate a forward iteration of the elements of the list."""
    cursor = self.first()
    while cursor is not None:
        yield cursor._element
        cursor = self.after(cursor)

def __len__(self):
    """Return the number of elements in the list."""
    return self._size

def is_empty(self):
    """Return True if list is empty."""
    return self._size == 0
```


单向链表的实现

```
# ----- mutators -----  
  
def add_first(self, e):  
    """Insert element e at the front of the list and return new position."""  
    newest = self._Node(e, self._head)  
    self._head = newest  
    self._size += 1  
    return self._head  
  
def add_after(self, p, e):  
    """Insert element e into list after position p and return new position."""  
    newest = self._Node(e, p._next)  
    p._next = newest  
    self._size += 1  
    return p._next  
  
def insert(self, n, e):  
    """Insert element e as the n-th node of the list and return the new position."""  
    if n == 0:  
        return self.add_first(e)  
    else:  
        cur = self.get_node(n - 1)  
        return self.add_after(cur, e)
```

单向链表的实现

```
def delete(self, n):
    """Remove and return the n-th node and return its element."""
    if n >= self._size:
        raise IndexError('Index out of bound!')
    if n == 0:
        cur = self._head
        self._head = self._head._next
    else:
        cur = self.get_node(n - 1)
        pre = cur
        cur = cur._next
        pre._next = cur._next
    element = cur._element
    cur._element = cur._next = None
    self._size -= 1
    return element
```

单向链表的实现

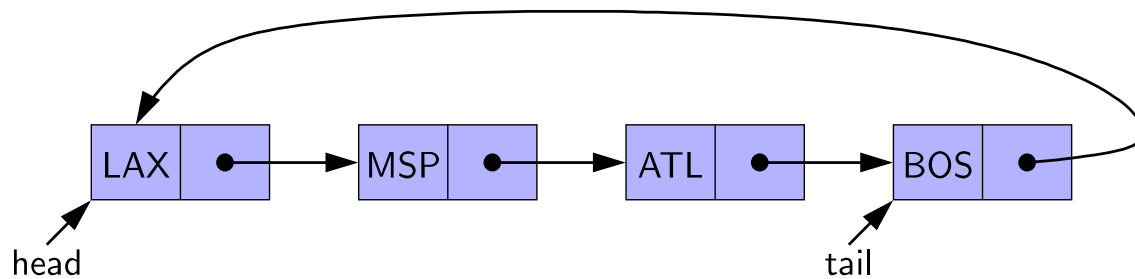
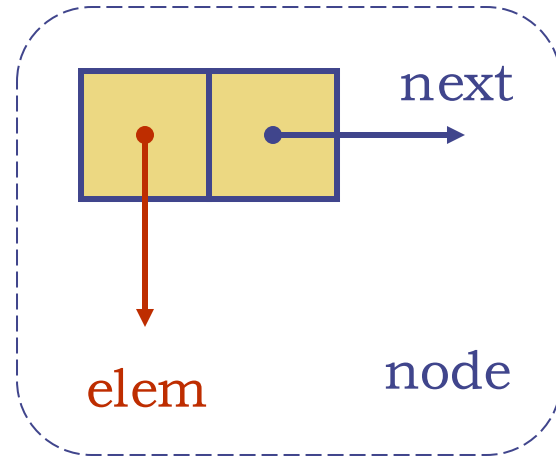
```
def replace_pos(self, p, e):
    """Replace the element at position p with e.
    Return the element formerly at position p.
    """
    old_value = p._element
    p._element = e
    return old_value

def replace_ind(self, n, e):
    """Replace the element at the n-th node with e.
    Return the element formerly at the n-th node.
    """
    p = self.get_node(n)
    return self.replace_pos(p, e)
```



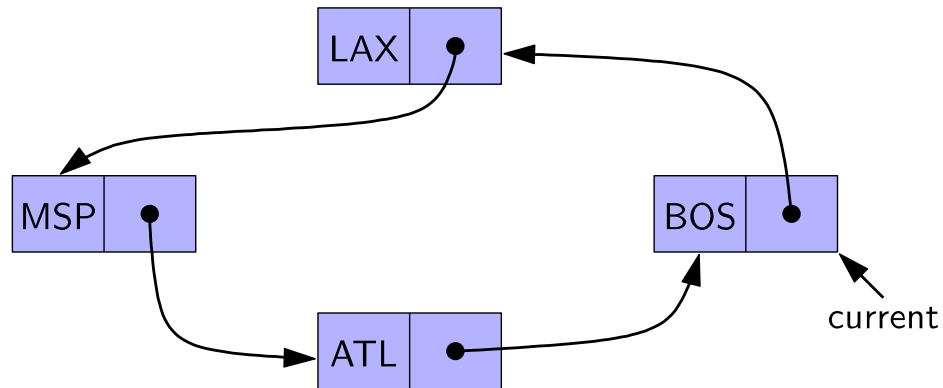
循环链表

- 循环链表是由单向链表的尾部节点的next指针指向头节点得到的
- 每个节点储存：
 - 元素值
 - 指向下一节点的指针



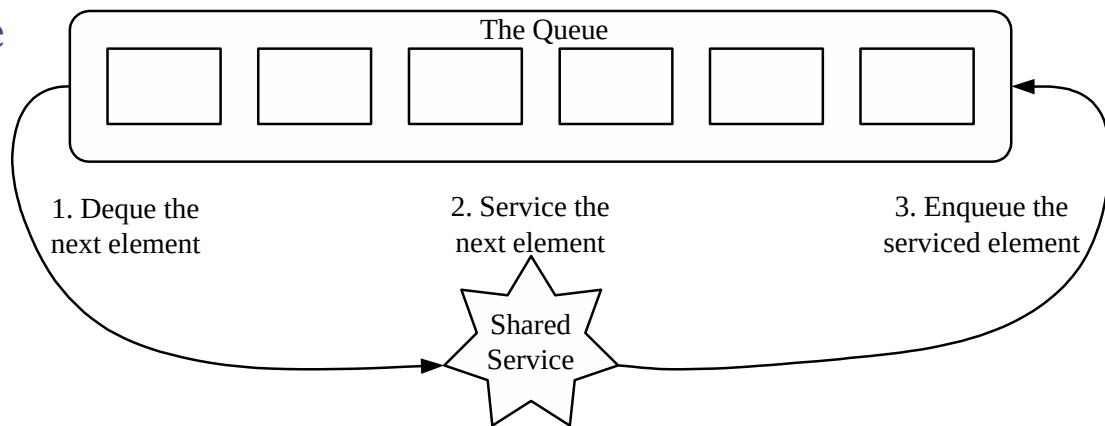
循环链表

- 循环链表为循环数据集提供了一个更通用的模型
- 我们可以使用current来表示当前结点



循环调度

- 考虑一个**循环调度**程序，以循环的方式遍历每一个元素的集合，并通过执行一个给定的动作为元素进行“服务”
- 例：循环调度常用于为同一计算机上多个并行的应用程序分配CPU时间片
- 使用普通队列Q，在其上反复执行以下步骤，即可实现循环调度：
 1. $e = Q.dequeue()$
 2. Service element e
 3. $Q.enqueue(e)$

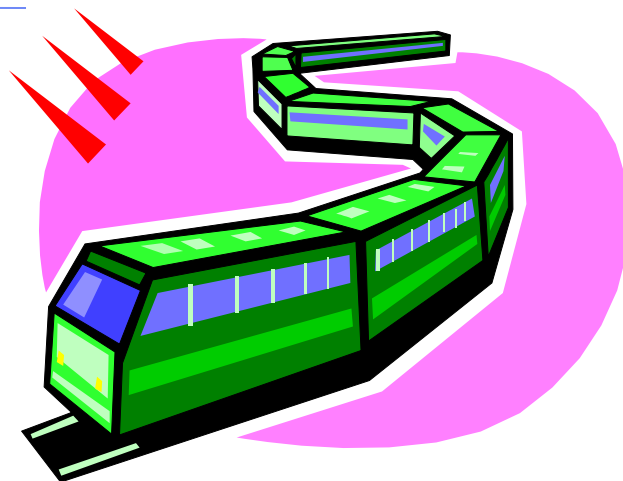


循环调度

- 使用循环链表可以方便地实现循环调度功能
- 在代码实现中，通过一个附加的方法rotate()将队头元素移动到队尾
- 可通过重复执行以下步骤有效地实现循环调度算法：
 1. Service element `Q.front()`
 2. `Q.rotate()`

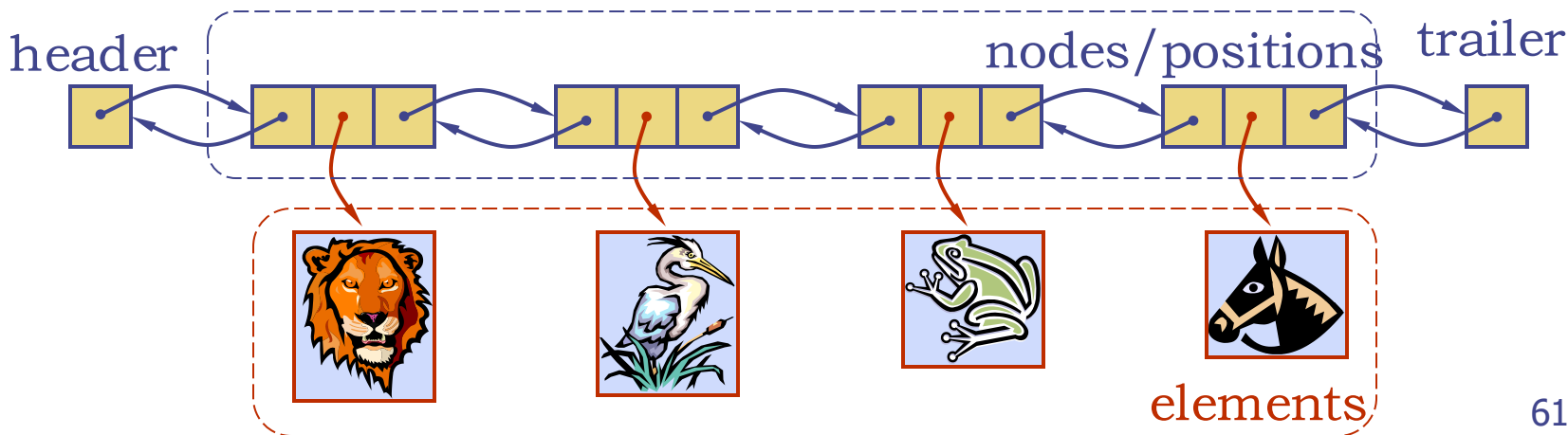
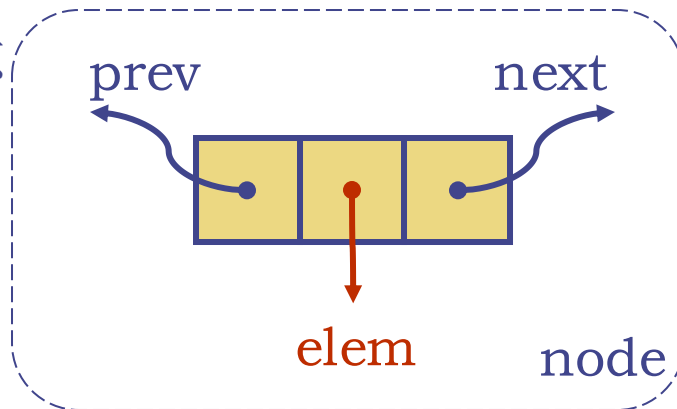


双向链表



双向链表

- 双向链表的每个节点都维护指向其先驱节点和后继节点的引用，有更好的对称性
- 双向链表的每个节点存储：
 - 元素值
 - 指向后继节点的指针
 - 指向先驱节点的指针
- 双向链表在链表头尾分别追加头节点 (header) 和尾节点 (trailer)



双向链表的ADT

ADT Doubly Linked List {

数据对象: $L = \{L_1, L_2, \dots, L_N\}$

基本操作:

$L.first()$: 返回 L 中第一个元素的位置, 如 L 为空则返回None

$L.last()$: 返回 L 中最后一个元素的位置, 如 L 为空则返回None

$L.before(p)$: 返回 L 中 p 紧邻的前面元素的位置, 如 p 为第一个位置则返回None

$L.after(p)$: 返回 L 中 p 紧邻的后面元素的位置, 如 p 为最后一个位置则返回None

$L.is_empty()$: 如 L 为空则返回True

$len(L)$: 返回列表元素个数

$iter(L)$: 返回列表元素的前向迭代器

双向链表的ADT

L.get_node(n): 找到L的第n个节点，并返回其位置

L.get_pos_element(p): 返回L中位置p处的元素值

L.add_first(e): 在L的前面插入新元素e，返回新元素的位置

L.add_last(e): 在L的后面插入新元素e，返回新元素的位置

L.add_before(p, e): 在L中位置p之前插入一个新元素e，返回新元素的位置

L.add_after(p, e): 在L中位置p之后插入一个新元素e，返回新元素的位置

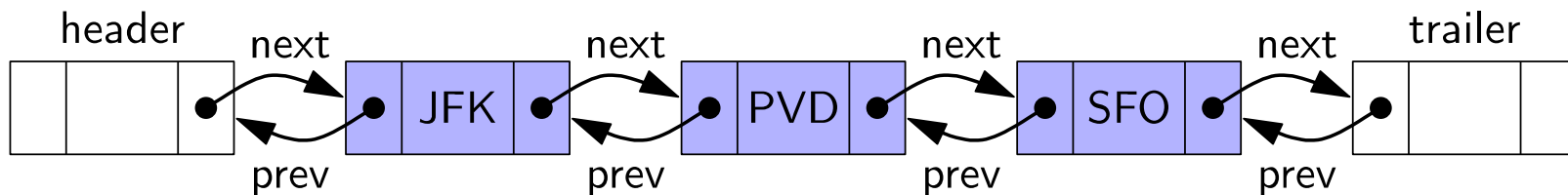
L.replace(p, e): 用元素e替代位置p处的元素，返回之前该位置的元素

L.delete(p): 删除并返回L中位置p处的元素，并使该位置无效

} ADT Doubly Linked List

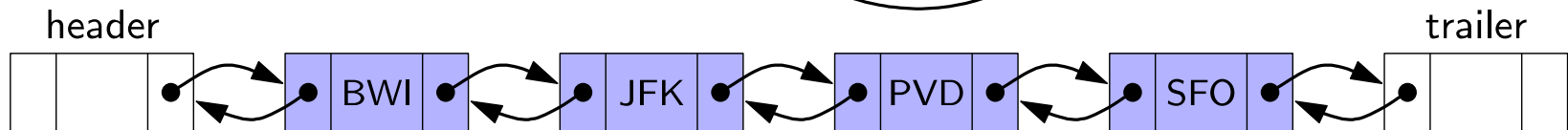
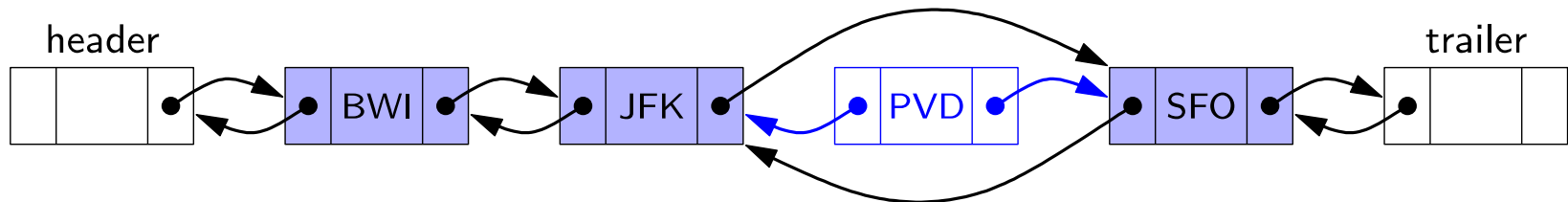
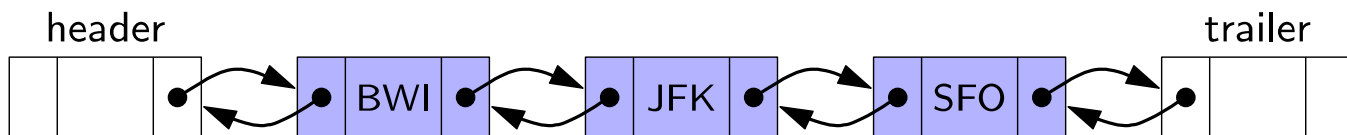
双向链表

- 操作双向链表的边界时，为了避免一些特殊情况，在链表两端都追加节点是很有用的：头节点（header）和尾节点（trailer）
- 这些特定的节点被称为哨兵（sentinel）。这些节点中不存储元素值
- 初始化双向链表时，需要创建头节点和尾节点，并让头节点的next指向尾节点，尾节点的prev指向头节点

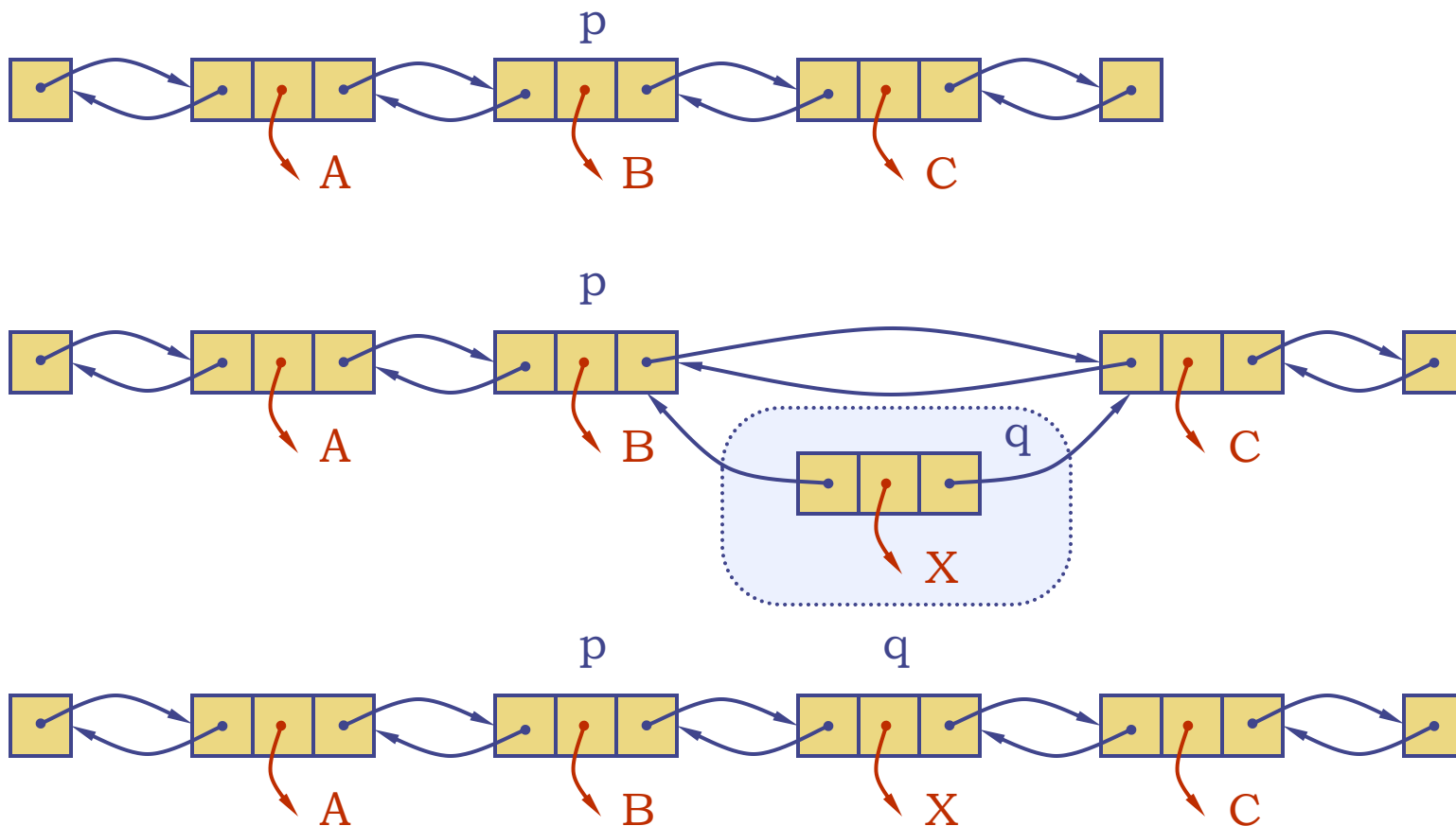


双向链表的插入

- 在节点p后插入新节点：
 - 创建一个新节点，将新元素放入新节点
 - 新元素的prev指向p，next指向p的后继节点
 - p的后继节点的prev指向新节点，p的next指向新节点

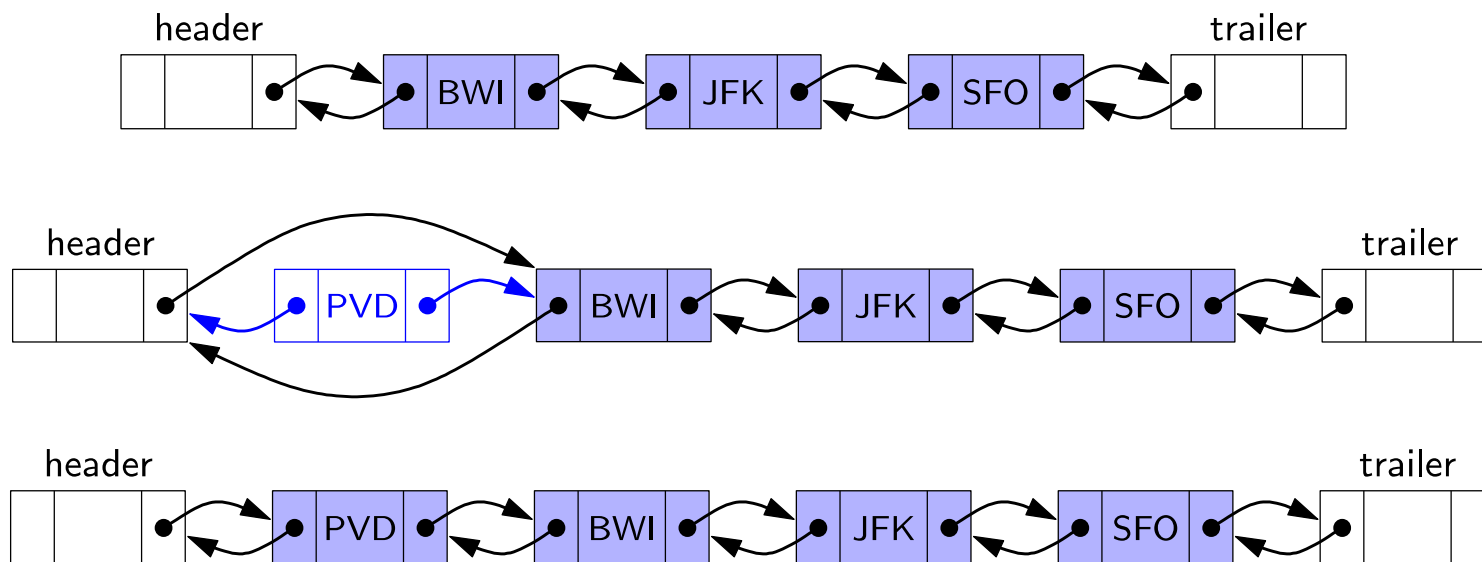


双向链表的插入



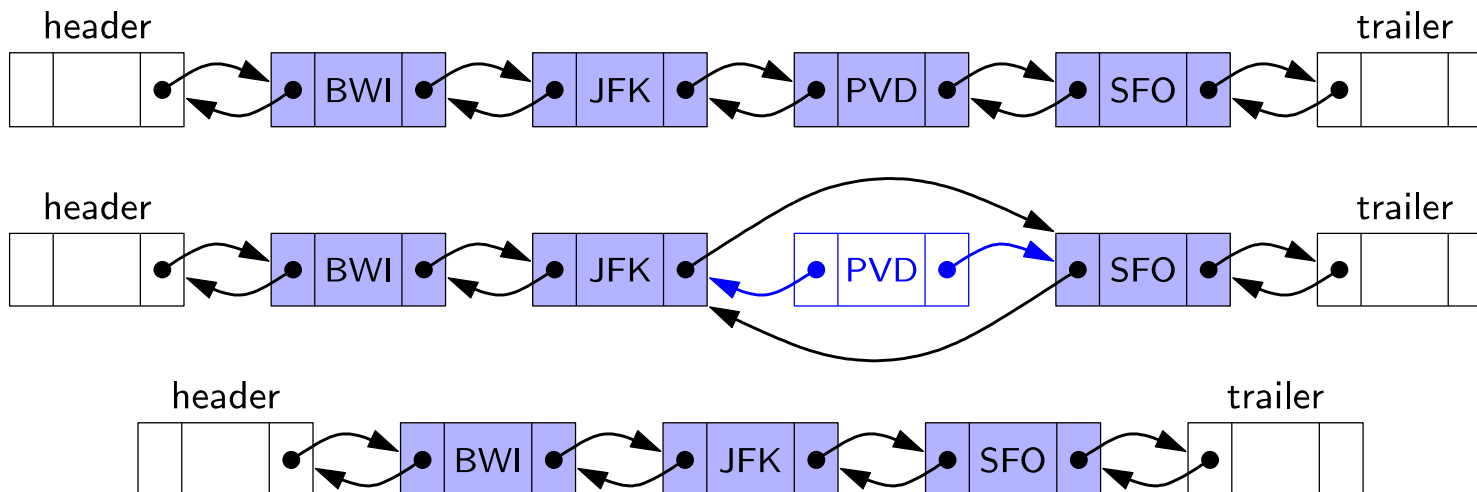
双向链表的插入

- 在链表头插入一个元素：

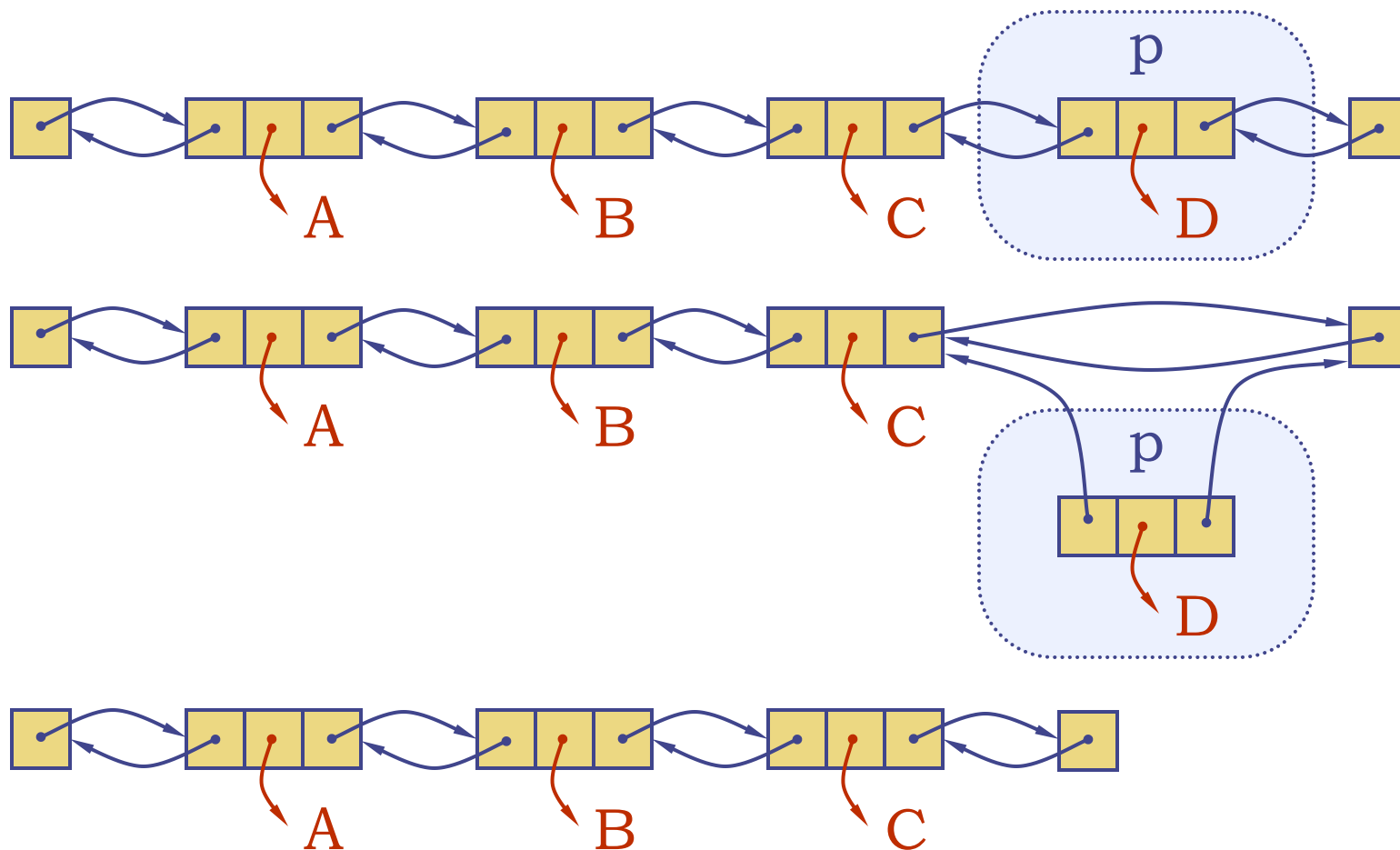


双向链表的删除

- 删除节点p:
 - p的先驱节点的next指向p的后继节点，p的后继节点的prev指向p的先驱节点
 - 释放节点p



双向链表的删除



双向链表的节点类

```
class _Node:
    """Lightweight, nonpublic class for storing a doubly linked node."""
    __slots__ = '_element', '_prev', '_next'      # streamline memory

    def __init__(self, element, prev, next):      # initialize node's fields
        self._element = element                 # user's element
        self._prev = prev                       # previous node reference
        self._next = next                       # next node reference
```

双向链表的实现

```
class DoublyLinkedList:
    """A base class providing a doubly linked list representation."""

    #----- nested _Node class -----
    # nested _Node class
    class _Node:
        """Lightweight, nonpublic class for storing a doubly linked node."""
        __slots__ = '_element', '_prev', '_next'      # streamline memory

        def __init__(self, element, prev, next):      # initialize node's fields
            self._element = element                  # user's element
            self._prev = prev                         # previous node reference
            self._next = next                         # next node reference
```

双向链表的实现

```
#----- list constructor -----

def __init__(self):
    """Create an empty list."""
    self._header = self._Node(None, None, None)
    self._trailer = self._Node(None, None, None)
    self._header._next = self._trailer          # trailer is after header
    self._trailer._prev = self._header          # header is before trailer
    self._size = 0                             # number of elements

#----- accessors -----

def first(self):
    """Return the first position in the list (or None if list is empty)."""
    return self._header._next

def last(self):
    """Return the last position in the list (or None if list is empty)."""
    return self._trailer._prev
```

双向链表的实现

```
def before(self, p):
    """Return the position just before position p (or None if p is first)."""
    return p._prev

def after(self, p):
    """Return the position just after position p (or None if p is last)."""
    return p._next

def get_node(self, n):
    """Return the n-th node (position) of the list."""
    if n >= self._size:
        raise IndexError('Index out of bound!')
    cur = self._head
    for k in range(n + 1):
        cur = cur._next
    return cur

def get_pos_element(self, p):
    """Return the element at position p."""
    return p._element
```

双向链表的实现

```
def __iter__(self):
    """Generate a forward iteration of the elements of the list."""
    cursor = self.first()
    while cursor is not None:
        yield cursor.element()
        cursor = self.after(cursor)

def __len__(self):
    """Return the number of elements in the list."""
    return self._size

def is_empty(self):
    """Return True if list is empty."""
    return self._size == 0

#----- mutators -----
def _insert_between(self, e, predecessor, successor):
    """Add element between existing nodes and return new position."""
    newest = self._Node(e, predecessor, successor) # linked to neighbors
    predecessor._next = newest
    successor._prev = newest
    self._size += 1
    return newest
```


双向链表的实现

```
def add_first(self, e):
    """Insert element e at the front of the list and return new position."""
    return self._insert_between(e, self._header, self._header._next)

def add_last(self, e):
    """Insert element e at the back of the list and return new position."""
    return self._insert_between(e, self._trailer._prev, self._trailer)

def add_before(self, p, e):
    """Insert element e into list before position p and return new position."""
    return self._insert_between(e, p._prev, p)

def add_after(self, p, e):
    """Insert element e into list after position p and return new position."""
    return self._insert_between(e, p, p._next)
```

双向链表的实现

```
def delete(self, p):
    """Remove and return the element at position p."""
    predecessor = p._prev
    successor = p._next
    predecessor._next = successor
    successor._prev = predecessor
    self._size -= 1
    element = p._element                                # record deleted element
    p._prev = p._next = p._element = None              # deprecate node
    return element                                       # return deleted element

def replace(self, p, e):
    """Replace the element at position p with e.

    Return the element formerly at position p.
    """
    old_value = p._element                                # temporarily store old element
    p._element = e                                         # replace with new element
    return old_value                                       # return the old element value
```

ANY
QUESTIONS
?

www.rekrutitn.com

ReKrutitn.com

数组vs链表

- 数组的优点：
 - 实现方式直接，几乎不会出错
 - 提供时间复杂度为 $O(1)$ 的基于整数索引的访问元素（随机访问）的方法
 - 通常，时间复杂度相同时，数组比链表运行时间的常数因子更小
 - 不需要存储指向下一个元素的地址
- 数组的缺点：
 - 插入、删除需移动大量元素
 - 表长变化不够灵活，可能存在存储空间的大量浪费

数组vs链表

- 链表的优点：
 - 链表支持在任意位置进行时间复杂度为 $O(1)$ 的插入和删除操作
 - 已知操作位置时，链表提供最坏情况下的操作时间界限
 - 表长易变化，存储空间利用灵活
- 链表的缺点：
 - 实现、使用方式容易出错
 - 基于整数索引的访问方式（随机访问）速度很慢
 - 存在指向下一个元素的地址开销

Enjoy